

Database System Engineering and Distributed Backend Development

Course Code: 25CS1302E

Practise-2

2520030098

Gutha Sushma Sree

Section – 3

Create the database:

```
CREATE DATABASE IF NOT EXISTS bookflow6_db;
```

Output:

```
✓ 1 14:42:17 CREATE DATABASE IF NOT EXISTS bookflow6_db
```

Use database:

```
USE bookflow6_db;
```

Output:

```
✓ 2 14:44:56 USE bookflow6_db
```

Create table:

```
CREATE TABLE orders (  
    ord_no INT PRIMARY KEY,  
    purch_amt DECIMAL(10,2),  
    ord_date DATE,  
    customer_id INT,  
    salesman_id INT  
);
```

Output:

```
✓ 3 14:46:13 CREATE TABLE orders ( ord_no INT PRIMARY KEY, purch_amt DECIMAL(10,2), or
```

Insert data:

```

INSERT INTO orders VALUES
(70001,150.50,'2012-10-05',3005,5002),
(70009,270.65,'2012-09-10',3001,5005),
(70002,65.26,'2012-10-05',3002,5001),
(70004,110.50,'2012-08-17',3009,5003),
(70007,948.50,'2012-09-10',3005,5002),
(70005,2400.60,'2012-07-27',3007,5001),
(70008,5760.00,'2012-09-10',3002,5001),
(70010,1983.43,'2012-10-10',3004,5006),
(70003,2480.40,'2012-10-10',3009,5003),
(70012,250.45,'2012-06-27',3008,5002),
(70011,75.29,'2012-08-17',3003,5007),
(70013,3045.60,'2012-04-25',3002,5001);
SELECT * FROM orders;

```

Output:

	ord_no	purch_amt	ord_date	customer_id	salesman_id
▶	70001	150.50	2012-10-05	3005	5002
	70002	65.26	2012-10-05	3002	5001
	70003	2480.40	2012-10-10	3009	5003
	70004	110.50	2012-08-17	3009	5003
	70005	2400.60	2012-07-27	3007	5001
	70007	948.50	2012-09-10	3005	5002
	70008	5760.00	2012-09-10	3002	5001
	70009	270.65	2012-09-10	3001	5005
	70010	1983.43	2012-10-10	3004	5006
	70011	75.29	2012-08-17	3003	5007
	70012	250.45	2012-06-27	3008	5002
	70013	3045.60	2012-04-25	3002	5001
*	NULL	NULL	NULL	NULL	NULL

Where Clause:

```

SELECT * FROM orders;
SELECT *
FROM orders
WHERE purch_amt > 2000;

```

Output:

	ord_no	purch_amt	ord_date	customer_id	salesman_id
▶	70003	2480.40	2012-10-10	3009	5003
	70005	2400.60	2012-07-27	3007	5001
	70008	5760.00	2012-09-10	3002	5001
	70013	3045.60	2012-04-25	3002	5001
*	NULL	NULL	NULL	NULL	NULL

Where Clause:

```
SELECT *  
FROM orders  
WHERE salesman_id=5001;
```

Output:

	ord_no	purch_amt	ord_date	customer_id	salesman_id
▶	70002	65.26	2012-10-05	3002	5001
	70005	2400.60	2012-07-27	3007	5001
	70008	5760.00	2012-09-10	3002	5001
	70013	3045.60	2012-04-25	3002	5001
*	NULL	NULL	NULL	NULL	NULL

Order By:

```
SELECT *  
FROM orders  
ORDER BY purch_amt DESC;
```

Output:

	ord_no	purch_amt	ord_date	customer_id	salesman_id
▶	70008	5760.00	2012-09-10	3002	5001
	70013	3045.60	2012-04-25	3002	5001
	70003	2480.40	2012-10-10	3009	5003
	70005	2400.60	2012-07-27	3007	5001
	70010	1983.43	2012-10-10	3004	5006
	70007	948.50	2012-09-10	3005	5002
	70009	270.65	2012-09-10	3001	5005
	70012	250.45	2012-06-27	3008	5002
	70001	150.50	2012-10-05	3005	5002
	70004	110.50	2012-08-17	3009	5003
	70011	75.29	2012-08-17	3003	5007
	70002	65.26	2012-10-05	3002	5001
*	NULL	NULL	NULL	NULL	NULL

Order By

```
SELECT *  
FROM orders  
ORDER BY ord_date;
```

Output:

	ord_no	purch_amt	ord_date	customer_id	salesman_id
▶	70013	3045.60	2012-04-25	3002	5001
	70012	250.45	2012-06-27	3008	5002
	70005	2400.60	2012-07-27	3007	5001
	70004	110.50	2012-08-17	3009	5003
	70011	75.29	2012-08-17	3003	5007
	70007	948.50	2012-09-10	3005	5002
	70008	5760.00	2012-09-10	3002	5001
	70009	270.65	2012-09-10	3001	5005
	70001	150.50	2012-10-05	3005	5002
	70002	65.26	2012-10-05	3002	5001
	70003	2480.40	2012-10-10	3009	5003
	70010	1983.43	2012-10-10	3004	5006
★	NULL	NULL	NULL	NULL	NULL

Sum()

```
SELECT SUM(purch_amt) AS total_revenue
FROM orders;
```

Output:

	total_revenue
▶	17541.18

Avg()

```
SELECT AVG(purch_amt) AS average_order
FROM orders;
```

Output:

	average_order
▶	1461.765000

max():

```
SELECT MAX(purch_amt) AS highest_order
FROM orders;
```

Output:

	highest_order
▶	5760.00

Count():

```
SELECT COUNT(*) AS total_orders
FROM orders;
```

Output:

	total_orders
▶	12

Group By:

```
SELECT salesman_id,
       SUM(purch_amt) AS total_sales
FROM orders
GROUP BY salesman_id;
```

Output:

	salesman_id	total_sales
▶	5002	1349.45
	5001	11271.46
	5003	2590.90
	5005	270.65
	5006	1983.43
	5007	75.29

Group By:

```
SELECT customer_id,
       SUM(purch_amt) AS total_purchase
FROM orders
GROUP BY customer_id;
```

Output:

	customer_id	total_purchase
▶	3005	1099.00
	3002	8870.86
	3009	2590.90
	3007	2400.60
	3001	270.65
	3004	1983.43
	3003	75.29
	3008	250.45

Group By + Max():

```
SELECT customer_id,
       MAX(purch_amt) AS highest_purchase
FROM orders
GROUP BY customer_id;
```

Output:

	customer_id	highest_purchase
▶	3005	948.50
	3002	5760.00
	3009	2480.40
	3007	2400.60
	3001	270.65
	3004	1983.43
	3003	75.29
	3008	250.45

Group By + Having

```
SELECT salesman_id,  
       SUM(purch_amt) AS total_sales  
FROM orders  
GROUP BY salesman_id  
HAVING SUM(purch_amt) > 3000;
```

Output:

	salesman_id	total_sales
▶	5001	11271.46

Group By + Having:

```
SELECT customer_id,  
       COUNT(*) AS total_orders  
FROM orders  
GROUP BY customer_id  
HAVING COUNT(*) > 1;
```

Output:

	customer_id	total_orders
▶	3005	2
	3002	3
	3009	2

Group By + Having + Order By:

```
SELECT customer_id,  
       SUM(purch_amt) AS total_purchase  
FROM orders  
GROUP BY customer_id  
HAVING SUM(purch_amt) > 1000  
ORDER BY total_purchase DESC;
```

Output:

	customer_id	total_purchase
▶	3002	8870.86
	3009	2590.90
	3007	2400.60
	3004	1983.43
	3005	1099.00

Between + Having:

```
SELECT customer_id,  
       MAX(purch_amt) AS max_purchase  
FROM orders  
GROUP BY customer_id  
HAVING MAX(purch_amt) BETWEEN 2000 AND 6000;  
|
```

Output:

	customer_id	max_purchase
▶	3002	5760.00
	3009	2480.40
	3007	2400.60

Count + Having:

```
SELECT salesman_id,  
       COUNT(*) AS total_orders  
FROM orders  
GROUP BY salesman_id  
HAVING COUNT(*) >= 2;  
.
```

Output;

	salesman_id	total_orders
▶	5002	3
	5001	4
	5003	2

Max + Group By + Having:

```
SELECT salesman_id,  
       COUNT(*) AS total_orders  
FROM orders  
GROUP BY salesman_id  
HAVING COUNT(*) >= 2;  
|
```

Output:

	salesman_id	total_orders
▶	5002	3
	5001	4
	5003	2